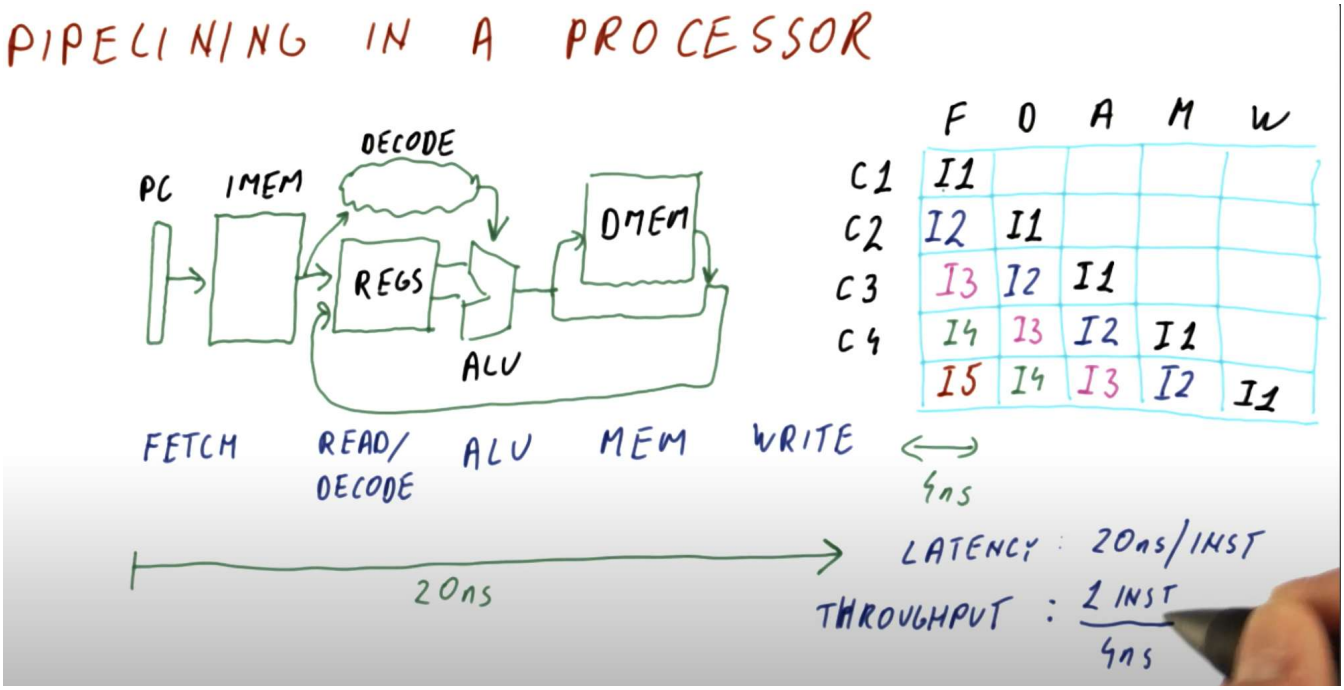
HPCA Chapter 3

Tuesday, March 18, 2025 8:41 PM

Vid 59 : In this lesson we will review pipelining in a way that sets stage for more advanced topics in Computer Architecture.

Vid 60 : Pipelining example from real life

Vid 61:



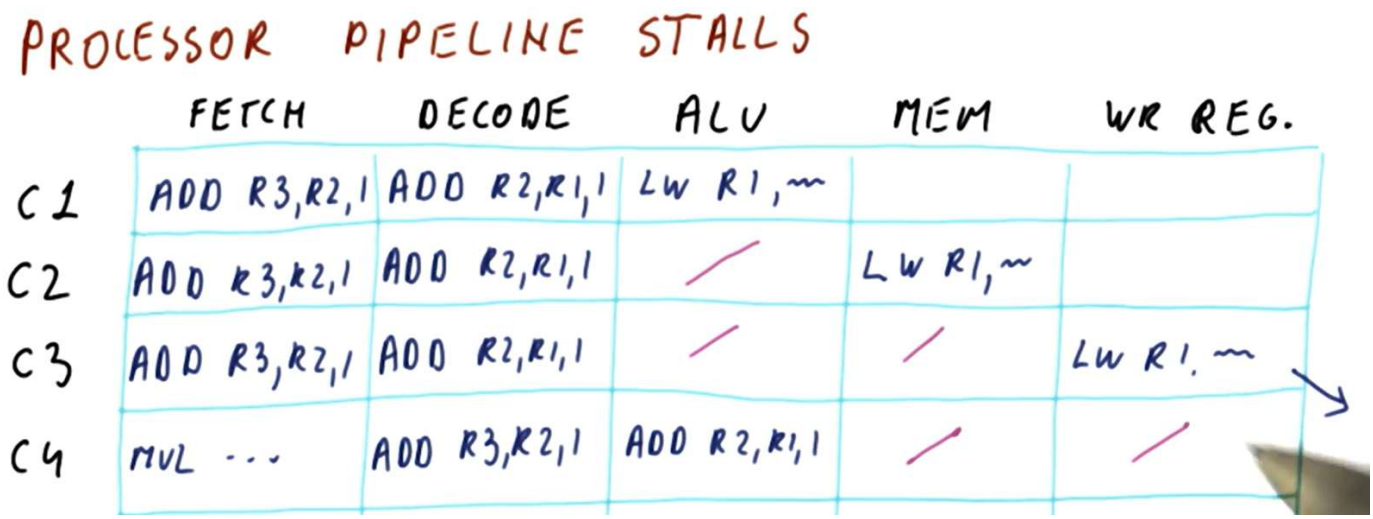
Shown above is a five stage processor without pipeline, each instruction would take 20ns to complete, but with pipelining, we have the same latency but instead of we getting one output every 20ns, we get one output every 4ns.

Vid 62 and 63 : Laundry Pipelining Quiz and sol

Vid 64 & 65 : Instruction Pipelining Quiz
We have a procesor with 5 stage pipeline, and we have 10 instructions, :
How many cycles will this take without pipelining: 50 cycles
with pipelining: 5 + 1*9 = 14 cycles

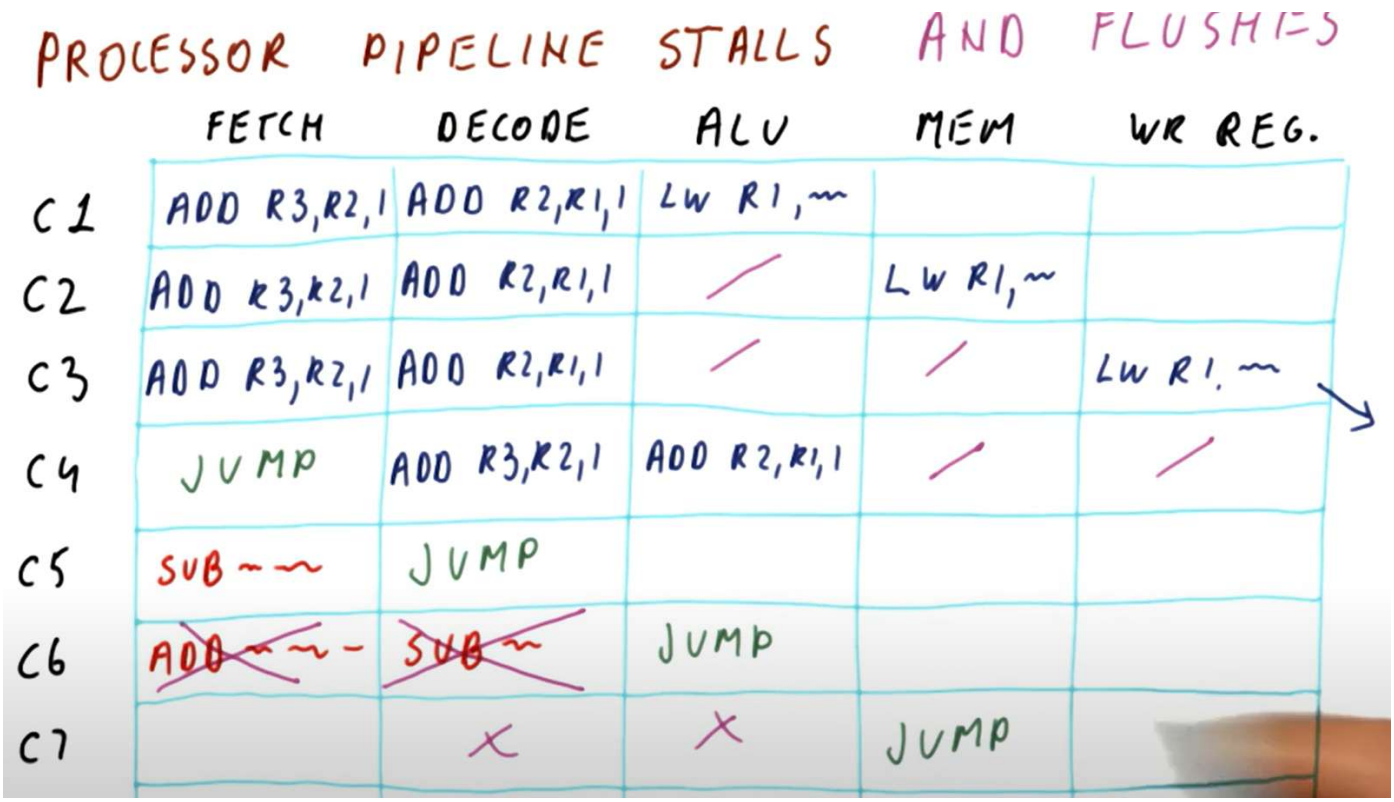
Vid 66 : Pipeline CPI is greater than or equal to 1, but usually its less than 1 due to pipeline stalls.

Vid 67: How pipeline stalls occur



Assuming LW (load from mem to our regs) and reading the same register can happen in same cycle(like loading in to r1 from mem is in 1st half of clock cycle, and reading it is being done in the second half of the clock cycle), we have two cycle stalls.

Vid 68: Pipeline flushes are gonna be an additional source of having CPI greater than 1, in this case, we figure out the right address to fetch from at the end of alu stage and then from next clock we fetch from the right location, but all instructions fetched from until now, are just wasted so they are made invalid and this bubble propagates.



VId 69: Branch and jump problems that we have seen are nothing but Control Dependencies
say we have code

ADD r1, r1, r2
BEQ r1, r3, Label
<p title = "code section 1">

```
ADD r2, r3, r4
SUB r5, r6, r8
.
.
.
```
** **

Label:

** **

```
MUL r5, r6, r8
.
.
.
```
** **

Here, code sections 1 &2 have a control dependency on BEQ instruction . their execution depends on the result produced by BEQ instruction. So we don't know whether we should be fetching these instructions until the branch is resolved.
say we have 20% of instructions are branches in a given code, and slightly more than half of them are taken branches what happens to cpi? 1+ 0.1*2 = 1.2 , 2 stages are flushed (its the penality for guessing the branch wrong) if we encounter a taken branch. If we have more branches as taken and more penality for fetching wrong instructions , our cpi is going to be higher. In modern procesors we have deep pipelines and we have branch resolution at some stages into the pipeline, so we need to have good prediction of branches to keep our cpi low.

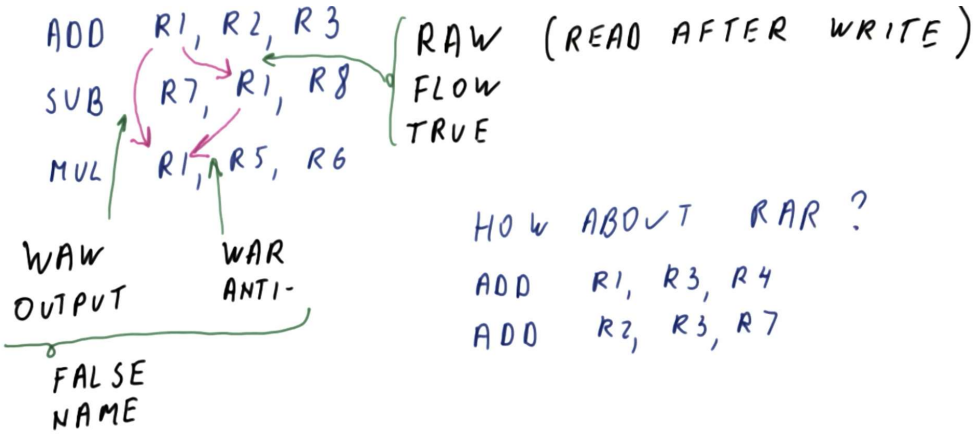
Vid 70 & 71 : Control Dependency Quiz:

- 25% of instructions are taken branch/jump
- 10 stage pipeline
- correct target for branch/jump is computed in 6th stage
- other than branches and jumps everything flows smoothly as far as CPI is concerned

Actual CPI is: 1 + 0.25*5 = 2.25

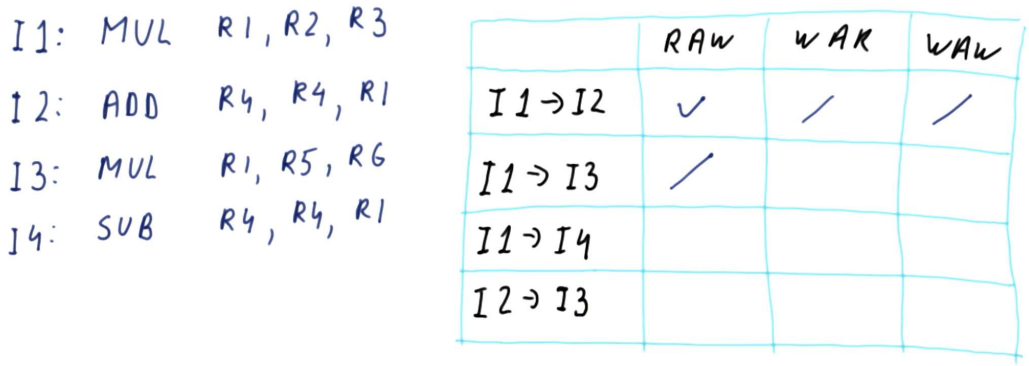
Vid 72: Data Dependencies

We can have data dependencies also that can stall pipeline..



Note that RAR is not a dependence, because reading doesn't modify the data

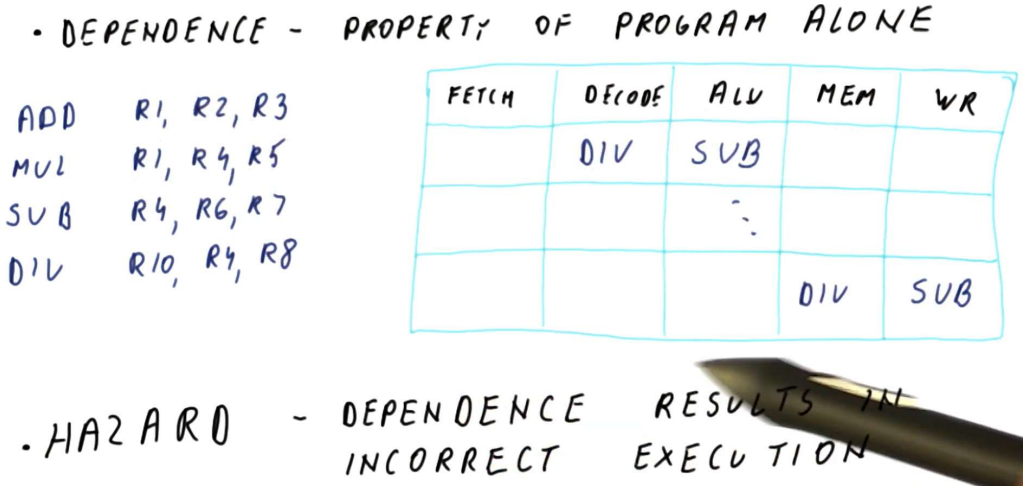
Vid 73 : Data depencies quiz



2nd row: waw
3rd row: ~~raw~~
4th row: war

Vid 75: Dependencies vs. hazards

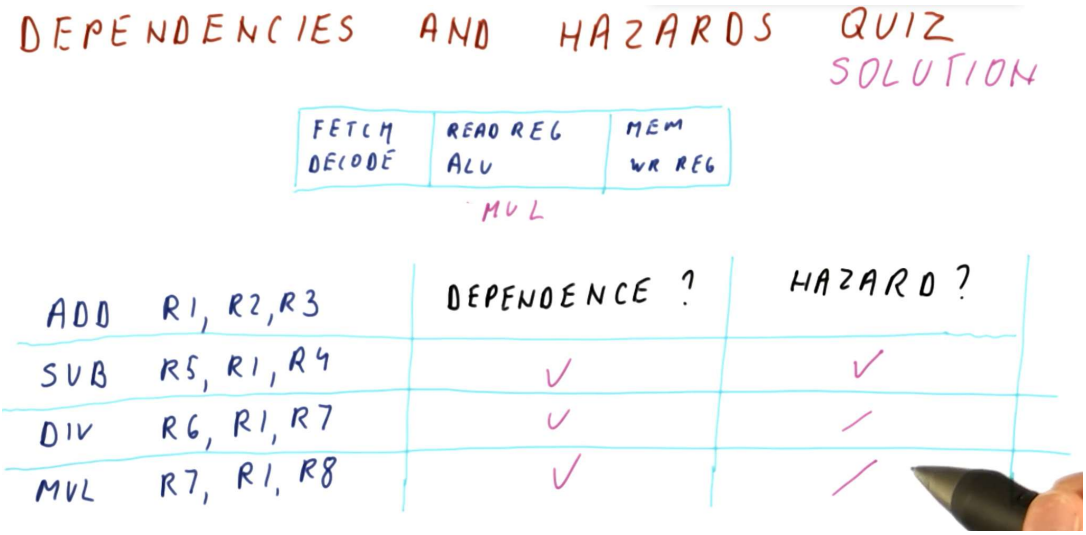
In a pipeline, some dependencies cause problems, and some don't. See in the example, ADD and Mul have an output dependence, but its never a problem. Similarly, SUB after MUL is never a problem , but there is a WAR dependence. IN case of a true dependence, is a problem if we dont
stall and wait for the result to get available. The situation when a dependence can cause a problem is called a hazard. The hazard is the property of the program and the pipeline. Not all true dependencies are hazards.



So in the below case, the XOR is reading what MUL is producing and ADD is producing, but say MUL writes at end of cycle and XOR reads at start of cycle, so when MUL is writing that reg, we are reading the same reg, so we get the previous value. SO RAW dependencies are not hazards unless the distance between the producing and consuming instns is atleast 3 cycles in this pipeline.

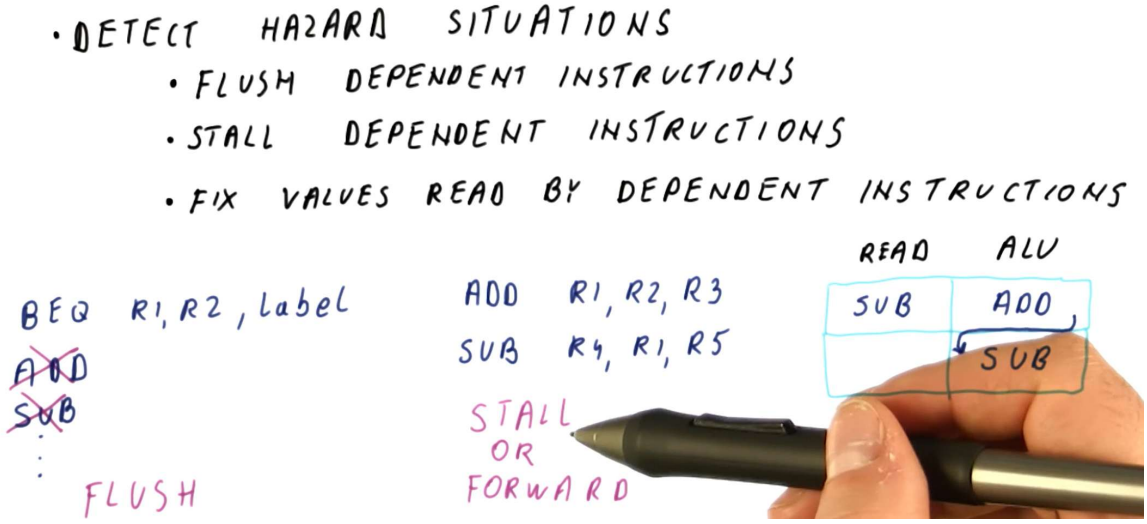


Vid 76 : Dependencies and Hazards Quiz



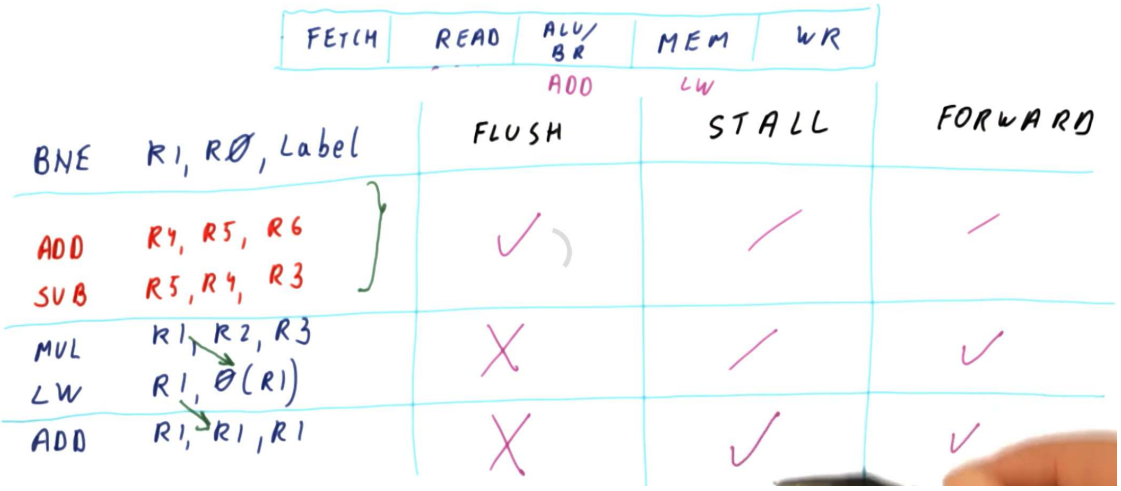
Vid 78 : Handling Hazards

If a control dependence becomes a hazard, we must flush the pipeline, we can't stall or fix.
In case of a data dependence, we can stall . FOr some data dependencies, we can forward the value produced to the consuming instn, in a way that fixes the read value. Forwarding will not always work, sometimes the value we need will be produced at later point in time, so at the point we need the value to compute, we still dont even have that value in the pipeline, At this time, we need to stall instead of forwarding.



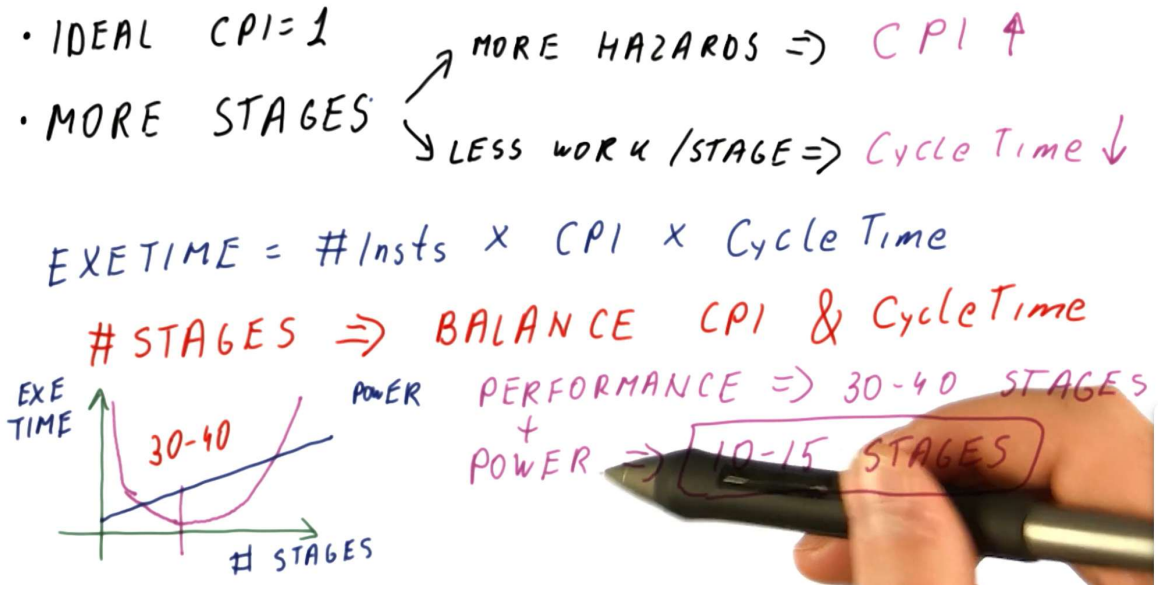
Vid 79 : Flushes stalls and forward quiz

The last add can be stalle 2 cycles, or instead we can stall only one cycle and at the end of this cycle we will have the required value in the pipeline, so we can forward the value.



Vid 81 : How many stages

THe balanced stages for modern processors are like 30-40 stages for performance. but considering power also, in real world we have like 10-15 stages, this balances good performance as well as power.



Vid 82 : Pipelines Outro

We now know how pipelines work, and how they need to balance the increasing frequency vs the stalls and flushes caused by hazards. Now that every processor uses pipelines, we will use this knowledge as we look at some of the more adv topics in the next few lessons.